

Decision Trees: The Complete Beginner's Guide

Sessions 1–4: From "What is a tree?" to "Random Forests"

Before We Start: The Big Picture

Imagine you're a college placement officer. Every year, 14 new students come to you, and you want to predict:
"Will this student get placed in a job or not?"

You have some info about each student:

- Their CGPA (grades)
- How many internships they did
- Whether they have backlogs (failed subjects)
- Their aptitude test score

A **decision tree** is a tool that looks at old students (whose outcomes you already know) and automatically builds a **flowchart of yes/no questions** that predicts placement for *new* students.

That's it. That's the whole idea. Everything below is just "how do we build that flowchart well, and how do we make it even better."

The 4 sessions build on each other like this:

Session	What it teaches	Simple one-line summary
1	What a tree IS and how it's built	"A tree is a chain of questions, built by picking the best question at each step."
2	HOW the tree decides which question is "best"	"Use math (entropy/Gini/MSE) to measure how good a question is."
3	How to stop a tree from getting too complicated	"A tree that's too detailed memorizes instead of learning — trim it."
4	How to use MANY trees together	"One tree can be wrong. Many different trees voting together are usually right."

Let's go one session at a time, very slowly.

SESSION 1: Decision Trees — Fundamentals & Greedy Construction

1.1 The Dataset We'll Use Everywhere

This exact table of 14 students is reused in **every single session**. Memorize its shape, not the numbers:

ID CGPA Internships Backlogs Aptitude Placed?

S1	8.6	2	No	73	Yes
S2	9.4	2	No	76	Yes
S3	9.0	2	No	81	Yes
S4	5.2	0	Yes	48	No
S5	5.3	0	Yes	44	No
S6	5.0	1	No	83	No
S7	6.2	1	Yes	63	No
S8	6.2	0	Yes	52	Yes
S9	8.7	2	Yes	77	Yes
S10	7.5	1	No	70	Yes
S11	6.7	1	No	96	Yes
S12	7.5	1	No	84	Yes
S13	6.8	0	Yes	48	No
S14	6.8	0	No	73	No

Totals: 8 students got Placed (Yes), 6 did not (No).

The **features** (inputs) are: CGPA, Internships, Backlogs, Aptitude.

The **target** (what we want to predict) is: Placed (Yes/No).

Since the target is a category (Yes/No), this is called a **classification** problem. (If instead we were predicting an exact salary number, that would be **regression** — more on this later.)

1.2 What Exactly IS a Decision Tree?

Simple definition: A decision tree is a series of yes/no (or threshold) questions arranged like an upside-down tree, where:

You start at the top

You answer a question

That answer sends you down a branch

You keep answering questions

Eventually you land on a final answer

Real-life analogy: Think of the game "20 Questions" or one of those doctor's decision charts: "*Do you have a fever? → Yes → Do you have a cough? → Yes → You probably have the flu.*" That's a decision tree.

The 4 parts of a tree (Anatomy)

Imagine this small tree:



Part	What it means	In our example
Root node	The very first question. Nothing comes before it.	"CGPA ≤ 7.15?"
Internal node	Any question that comes <i>after</i> the root, but is NOT the final answer	"Backlogs = Yes?"
Branch	The arrow showing which way you go based on your answer	The "Yes" or "No" arrows
Leaf node	The END of a path. This gives the actual prediction. No more questions after this.	"Placed" or "Not Placed"

 **Beginner tip:** Think of "leaf" literally like a leaf on a real tree — it's at the very tip/end of a branch, nothing grows after it.

How to "read" a tree for one specific student

Let's say a NEW student walks in with: CGPA = 8.2, Backlogs = No.

Using the tree above:

Start at root: "CGPA $\leq$ 7.15?" $\rightarrow$ Their CGPA is 8.2, so the answer is **No**.

Following the "No" branch takes us straight to a leaf: **Placed = Yes**.

Done! We never even looked at the Backlogs question, because that branch wasn't on our path.

 **Key beginner insight:** A single prediction only ever travels down **ONE path** from root to one leaf. It does NOT touch every node in the tree. Different students may use completely different paths.

1.3 How Is the Tree Actually Built? ("Greedy Top-Down Construction")

This is the most important idea in Session 1. Let's break the fancy term into two simple words:

"Top-Down" — meaning:

You start with **ALL** your data lumped together at the very top (the root), and you keep splitting it into smaller and smaller groups as you go down. You never start from the bottom.

Analogy: Imagine you have a huge pile of 14 student folders in one basket. You're going to sort them into smaller and smaller baskets, one question at a time.

"Greedy" — meaning:

At each step, the tree picks whichever question looks **best right now** — it does NOT think ahead about what might happen 2 or 3 questions later. It's a bit short-sighted, like a hungry person grabbing the biggest slice of pizza available *right now* without thinking about whether a smaller slice now might leave a better slice for later.

Why "greedy" instead of "perfectly optimal"? Because checking literally every possible tree combination for even a modest dataset would take forever (computationally impossible). Greedy is a shortcut: "just always make the locally best choice." It's fast and usually works well, but it's not mathematically guaranteed to build the single best possible tree in the universe.

The step-by-step construction loop

Start: Put all 14 students in one box (the root).

Test candidate questions: Try out different possible questions, e.g.:

"CGPA $\leq$ 7.15?"

"Internships $\leq$ 1?"

"Backlogs = Yes?"

"Aptitude $\leq$ 65?"

Score each candidate question: (Session 2 explains exactly HOW to score them — using entropy/Gini math.)

Pick the winner: Choose whichever question does the best job of separating the Placed students from the Not-Placed students.

Split: Divide your box of 14 students into two smaller boxes based on the answer to that question.

Repeat: For each new smaller box, if it's still a *mixture* of Yes and No, repeat steps 2–5 on just that smaller box.

Stop: When a box only contains ONE type of student (all Yes, or all No) — or some other stopping rule is hit (Session 3) — turn that box into a leaf.

Pure vs. Mixed nodes (very important vocabulary)

Pure node = every student inside has the SAME outcome. Example: a box with 6 students, all "Placed." Nothing more to decide — turn it into a leaf immediately.

Mixed node = the box has a blend of outcomes. Example: 2 Placed + 6 Not Placed. This box still needs more splitting because it's "confused" / uncertain.

Simple visual:



A "good" split is simply one where the resulting groups are **cleaner/purer** than what you started with.

1.4 Numerical Features vs. Categorical Features

Not all questions look the same, because not all columns in our data are the same TYPE.

Numerical features (numbers that can be compared, like CGPA, Aptitude, Internships)

The tree picks a **threshold** (a cutoff number) and asks: "*Is this value $\leq$ the threshold, or above it?*"

Example: "CGPA $\leq$ 7.15?" $\rightarrow$ splits into "CGPA is 7.15 or lower" vs. "CGPA is higher than 7.15."

How is the threshold (7.15) chosen? You sort all the CGPA values in the data from smallest to largest, and look for spots where the Yes/No label seems to naturally "flip." You take the midpoint between two neighboring values there. In our data, values 6.8 (No) and 7.5 (Yes) sit next to each other where the split matters most, so:

$$\text{threshold} = \frac{6.8 + 7.5}{2} = 7.15$$

Categorical features (categories/labels, like Backlogs)

The tree just directly asks about the category: "*Backlogs = Yes?*" $\rightarrow$ splits into "has backlogs" vs. "doesn't have backlogs." No threshold math needed — it's already a clean Yes/No.

1.5 Classification Trees vs. Regression Trees

So far we've been predicting Placed/Not Placed — a **category**. But decision trees can also predict **numbers**, like salary.

	Classification Tree	Regression Tree
What it predicts	A category/label	A number
Example	Placed / Not Placed	Salary package (e.g., ₹6.5 LPA)
What the leaf gives you	The most common class in that leaf	The AVERAGE of all values in that leaf
Typical "goodness" measure	Gini or Entropy	MSE (Mean Squared Error)
How we check accuracy	Accuracy, F1 score	RMSE, MAE

Beginner analogy: Classification is like sorting mail into "Local" vs "International" bins. Regression is like estimating exactly how much a package weighs.

The actual *process* of building the tree (top-down, greedy, split-and-repeat) is identical for both — only the *math used to judge a "good split"* changes.

1.6 When Does a Tree STOP Growing?

If you never told it to stop, a tree could keep splitting forever, right down to leaves with just 1 student each. That's bad (we'll see exactly why in Session 3). So there are stopping rules:

Natural stopping (the tree stops on its own):

The node is already pure (all one class) — nothing left to fix.

No possible split would improve things.

All remaining students literally have identical feature values (nothing left to distinguish them by).

Pre-pruning controls (limits YOU set beforehand):

Maximum depth reached (we'll set a limit on how many questions deep it can go)

Too few students left to bother splitting

A proposed split would create a leaf that's too tiny

We've hit the maximum number of leaves we allowed

The improvement from splitting further is too small to be worth it

(Session 3 dives deep into these — this is just a preview.)

1.7 Session 1 Summary (Quick Recap)

A decision tree = a learned sequence of feature-based questions that leads to a prediction.

It has a **root**, **internal nodes**, **branches**, and **leaves**.

It's built **top-down** (start with all data) and **greedily** (best choice at each step, no forward-planning).

Pure nodes (all one class) become leaves; **mixed** nodes get split further.

Numerical features use threshold questions (" $\leq$ some number?"); categorical features use direct category questions.

Classification trees predict categories; regression trees predict numbers.

Trees need stopping rules or they'll grow forever and overfit.

SESSION 2: Splitting Criteria & Worked Examples

Session 1 said "pick whichever question is best." Session 2 answers: **best according to WHAT math?**

2.1 The Problem We're Solving

At the root, we have 14 students: 8 Placed, 6 Not Placed. That's a "confused" mix. We want a NUMBER that tells us exactly *how* confused/mixed this group is — so that after we try a split, we can compare "how confused before" vs. "how confused after," and see if we actually improved things.

That number is called **impurity** (or "uncertainty"). Session 2 teaches you 3 ways to measure it:

Entropy (paired with Information Gain)

Gini Impurity

MSE (for regression only)

2.2 Entropy — Measuring "How Mixed Is This Group?"

The simple idea first (no formula yet)

Imagine 3 boxes of students:

Box A: 10 Placed, 0 Not Placed → completely one type. Zero confusion. "Entropy = 0."

Box B: 5 Placed, 5 Not Placed → a perfect 50/50 coin-flip. Maximum confusion. "Entropy = 1" (its highest possible value, for a 2-category problem).

Box C: 8 Placed, 2 Not Placed → mostly one type, but not entirely. Entropy is somewhere in between, closer to 0.

In plain English: Entropy answers "if I randomly grabbed one student from this box, how uncertain am I about whether they're Placed or Not Placed?" All-same-type = no uncertainty = entropy 0. Perfectly 50/50 = maximum uncertainty = entropy close to 1.

The formula

$$H(S) = - \sum p_i \log_2(p_i)$$

For our binary (2-category) case, this simplifies to:

$$H(S) = -p_1 \log_2(p_1) - p_0 \log_2(p_0)$$

Where p_1 = fraction of "Placed" students, p_0 = fraction of "Not Placed" students.

Don't panic about the $\log_2$ — you'll basically always use a calculator for this. What matters is you understand *what it represents*, not that you can compute logs by hand.

Worked Example: Entropy at the Root

Our root has 8 Placed and 6 Not Placed out of 14 total.

$$p_1 = \frac{8}{14} = 0.5714 \text{ (Placed)}$$

$$p_0 = \frac{6}{14} = 0.4286 \text{ (Not Placed)}$$

$$H(S) = -(0.5714) \log_2(0.5714) - (0.4286) \log_2(0.4286) = \boxed{0.9852}$$

What does 0.9852 mean? It's very close to 1 (the maximum), so our root is *almost* as confused/mixed as it could possibly be. This makes sense — 8 vs 6 is close to an even split. This number, 0.9852, is our **starting point** — the "impurity before we do anything."

2.3 Testing a Candidate Split: CGPA $\leq$ 7.15

Let's actually try splitting on this question and see if it helps.

Step 1 — Divide the 14 students:

Left child (CGPA $\leq$ 7.15): 8 students total $\rightarrow$ 2 Placed, 6 Not Placed

Right child (CGPA $>$ 7.15): 6 students total $\rightarrow$ 6 Placed, 0 Not Placed

Notice the right side is **completely pure** already! All 6 of those students got placed. 🎉

Step 2 — Compute entropy of EACH child separately:

Left child (2 Placed, 6 Not Placed out of 8):

$$H_{left} = - \left(\frac{2}{8} \right) \log_2 \left(\frac{2}{8} \right) - \left(\frac{6}{8} \right) \log_2 \left(\frac{6}{8} \right) = 0.8113$$

Right child (6 Placed, 0 Not Placed out of 6):

$$H_{right} = 0 \quad (\text{it's pure — no confusion at all})$$

Step 3 — Combine them into one "weighted" number.

Why "weighted"? Because the left group has 8 students and the right group has 6 — they're not equal sizes, so we can't just average the two entropy numbers directly. We weight each by how many students are in it (out of the total 14):

$$H(\text{children}) = \frac{8}{14}(0.8113) + \frac{6}{14}(0) = \boxed{0.4636}$$

Step 4 — Compare: did entropy go DOWN?

Before the split: 0.9852 After the split: 0.4636

Yes! It dropped a lot — meaning our split made the data much less confusing.

2.4 Information Gain — "How much did this split actually help?"

$$\text{Information Gain (IG)} = H(\text{parent}) - H(\text{children})$$

$$IG(\text{CGPA} \leq 7.15) = 0.9852 - 0.4636 = \boxed{0.5216}$$

In plain English: Information Gain is simply "how much confusion did we remove?" The bigger this number, the better the split. **Rule: pick the split with the HIGHEST Information Gain.**

Comparing several candidate questions at once

The tree doesn't just try one question — it tries ALL reasonable candidate questions and computes IG for each, then picks the winner:

Candidate Question Weighted Entropy After Split Information Gain

CGPA $\leq$ 7.15	0.4636	0.5216 ★ (BEST)
Aptitude $\leq$ 50	0.6642	0.3210
Internships $\leq$ 1.5	0.6935	0.2917
Backlogs = No/Yes	0.8571	0.1281

"CGPA ≤ 7.15?" wins — it has the highest Information Gain, so it becomes the ROOT question of our tree.

⚠ **Common beginner mistake:** Don't confuse "highest raw entropy" with "best split." You want the question that gives the **biggest drop/reduction** in entropy (highest IG), not the one that just happens to have a high number somewhere.

2.5 Gini Impurity — An Alternative to Entropy

Entropy uses logarithms, which are a bit slow to compute. **Gini Impurity** measures almost the exact same idea (how mixed is this group?) using a simpler formula — no logs needed.

$$Gini(S) = 1 - \sum p_i^2$$

For binary classification:

$$Gini(S) = 1 - (p_1^2 + p_0^2)$$

Gini = 0 → pure node (no mixing)

Gini is higher → more mixed (max value for binary is 0.5, vs entropy's max of 1)

Rule: LOWER weighted Gini after a split = better.

Worked Example: Gini at the Root

$$Gini(\text{root}) = 1 - \left[\left(\frac{8}{14} \right)^2 + \left(\frac{6}{14} \right)^2 \right] = \boxed{0.4898}$$

Gini after the CGPA ≤ 7.15 split

Node	Placed	Not Placed	Gini
Left (CGPA ≤ 7.15)	2	6	$1 - [(2/8)^2 + (6/8)^2] = 0.3750$
Right (CGPA > 7.15)	6	0	$1 - [1^2 + 0^2] = 0.0000$

Weighted Gini:

$$\frac{8}{14}(0.3750) + \frac{6}{14}(0) = \boxed{0.2143}$$

Impurity dropped from 0.4898 → 0.2143. Same conclusion as entropy: **this is a genuinely good split**, confirmed by a second, independent method.

Entropy vs. Gini — should you care which one is used?

	Entropy	Gini
Needs logarithms?	Yes	No — just squaring
Speed	Slower	Faster (this is why it's often the DEFAULT in real tools like scikit-learn)
Max value (binary)	1	0.5
Do they usually agree on the best split?	Mostly yes, occasionally they pick different splits	

Beginner takeaway: They're two different rulers measuring the same thing ("how mixed is this group"). You don't need to memorize both formulas perfectly — just understand that BOTH answer "is this split making the data cleaner?" and BOTH prefer bigger drops in impurity.

2.6 MSE — For When You're Predicting NUMBERS, Not Categories

Entropy and Gini only work when your target is a category (Yes/No). What if instead we wanted to predict an exact **salary number** for each student?

We can't use "probability of Yes" anymore — there's no "Yes" category. Instead, we use **Mean Squared Error (MSE)**:

$$MSE = \frac{1}{n} \sum (y_i - \bar{y})^2$$

In plain English:

Take the average of all values in the node ($\bar{y}$).

For each actual value, measure how far it is from that average, and square that distance (squaring makes big errors count extra, and removes negative signs).

Average all those squared distances.

A regression tree's leaf doesn't say "Yes" or "No" — it says "the predicted value is the AVERAGE of everyone in this leaf." A good split is one where the two child groups end up with LESS spread-out (more tightly clustered) values than the parent had.

Worked "teaching" example (using Placed=1, Not Placed=0 as if they were numbers)

Root: 8 students are "1" (Placed), 6 are "0" (Not Placed).

$$\text{Average} = \frac{8}{14} = 0.5714$$

$$MSE(\text{root}) = 0.2449$$

After splitting on $CGPA \leq 7.15$:

Node	Count of 1s	Count of 0s	Mean	MSE
$CGPA \leq 7.15$	2	6	0.25	0.1875
$CGPA > 7.15$	6	0	1.00	0.0000

$$\text{Weighted MSE} = \frac{8}{14}(0.1875) + \frac{6}{14}(0) = \boxed{0.1071}$$

Error dropped: $0.2449 \rightarrow 0.1071$. Once again — same conclusion, this is a strong, useful split! (Note: this is just a teaching demo to show the *mechanics* of MSE; in real life, you'd use MSE on an actual continuous number like salary, not a 1/0 stand-in for a category.)

2.7 Summary Table: Which Measure for Which Job?

Measure	Used for	Measures	Want it...
Entropy	Classification	Uncertainty/impurity	LOWER
Information Gain	Classification	How much entropy a split removes	HIGHER
Gini Impurity	Classification	Class impurity	LOWER
MSE	Regression	Squared prediction error	LOWER

2.8 Full Worked Example: The "Play Cricket" Dataset (ID3 Algorithm)

To see the WHOLE process from start to finish, here's a classic teaching dataset: should we play cricket today, based on weather?

Features: Outlook (Sunny/Overcast/Rainy), Temperature, Humidity, Windy. Target: Play (Yes/No).

Step 1 — Compute parent (root) entropy across all 14 days: $H(S) \approx 0.940$ (again, close to max — pretty mixed).

Step 2 — Test every feature's Information Gain:

Feature	Information Gain
Outlook	0.247 ★ (winner)
Humidity	0.152
Windy	0.048
Temperature	0.029

Step 3 — Outlook becomes the ROOT, splitting into 3 branches (since Outlook has 3 categories, not 2): Sunny, Overcast, Rainy.

Step 4 — Check each branch — is it pure or still mixed?

Overcast branch: turns out EVERY day here says "Yes, play cricket." Pure! → becomes an immediate leaf, no more questions needed.

Sunny branch: still a mix of Yes/No → needs another split. Re-run the same "test all remaining features, pick highest IG" process just on THIS smaller subset of days → **Humidity** wins here.

Rainy branch: still a mix → same process again → **Windy** wins here.

Step 5 — Final tree:



The big lesson here: this shows the tree-building process is **recursive** — meaning the exact same "measure impurity → test all candidates → pick best IG → split → check if pure → repeat" loop just runs over and over on smaller and smaller pieces of the data, until every branch ends in a pure leaf (or hits a stopping rule).

2.9 Session 2 Summary (Quick Recap)

Entropy measures how mixed/uncertain a group is (0 = pure, ~1 = maximally mixed).

Information Gain = entropy before split – entropy after split. Pick the split with the HIGHEST IG.

Gini Impurity is a faster alternative to entropy; same goal, pick the split with the LOWEST resulting Gini.

MSE does the same job as entropy/Gini but for regression (numeric) targets.

The tree tests ALL reasonable candidate splits at each node and picks whichever improves purity the most. This process repeats recursively on each new sub-group until nodes are pure or a stopping rule is hit.

SESSION 3: Tree Pruning — Controlling Decision-Tree Complexity

Session 1 built a tree. Session 2 explained how it picks good questions. Now: **what if the tree gets TOO good at fitting the training data — so good that it stops being useful for new students?**

3.1 The Core Problem: Overfitting

Simple story first: Imagine a teacher who creates a study guide by literally copying every single question from last year's exact exam, word for word, with the exact answers. If this year's exam has even slightly different questions, that "study guide" is useless — it memorized instead of learning general concepts.

A decision tree can do the same thing. If you let it keep splitting forever, it can create a leaf for practically every single individual student — a leaf that says "if CGPA is EXACTLY 7.5 AND Internships is EXACTLY 1 AND Aptitude is EXACTLY 84 → Placed." That's not a useful *pattern*, that's just re-stating one specific student's data. This is called **overfitting**.

Underfitting vs. Good Fit vs. Overfitting

Tree size	What happens	Training error	Error on NEW/unseen students	Verdict
Too small (very few questions)	Misses real, useful patterns	High	High	 Underfitting
Just right (moderate size)	Captures real patterns, ignores noise	Low	Low	 Good fit
Too big (way too many questions)	Memorizes exact training examples, including their randomness/noise	Very low	Can get WORSE	 Overfitting

 **Critical beginner insight:** A bigger, deeper tree will almost ALWAYS score better on the *training* data (the data it already saw) — that's not impressive, it's basically cheating by memorizing. What actually matters is how well it performs on students it has NEVER seen before (validation/test data). **Judge trees by unseen-data performance, not training accuracy.**

3.2 Bias-Variance: A Simple Way to Think About Tree Size

Two new words, explained simply:

Bias = how "rigid"/oversimplified your model's assumptions are. High bias = the tree is too simple and stubbornly ignores real patterns (this is underfitting).

Variance = how much your model's predictions would change if you trained it on a slightly different set of students. High variance = the tree is overly sensitive — tiny changes in training data cause wildly different trees (this is overfitting).

Tree type	Bias	Variance	Result
Very simple/shallow tree	HIGH bias	LOW variance	Underfits
Very deep/complex tree	LOW bias	HIGH variance	Overfits
Well-pruned tree	Slightly higher bias than deep tree MUCH lower variance		Best generalization

Analogy: Imagine trying to draw a line through scattered dots.

A perfectly straight ruler-line (too simple) will miss the curve of the data (high bias, underfits).

A wildly wiggly line that touches every single dot exactly (too complex) will look perfect on paper but be useless for predicting a new dot (high variance, overfits).

Something smooth and reasonably curved in between is what you actually want.

The rule of thumb: More pruning → simpler tree → higher bias, lower variance. Less pruning → deeper tree → lower bias, higher variance. Pruning is how we deliberately walk this dial to the sweet spot.

3.3 Pruning: The Fix for Overfitting

Pruning just means "cutting down (or preventing) unnecessary complexity in a tree, so it works better on new data." There are two families of approaches, and the difference is simply **WHEN** you apply the brakes:

3.4 Pre-Pruning: Stop the Tree BEFORE It Grows Too Much

Think of pre-pruning like telling a kid "you can only ask 5 questions before you have to guess" — you set the limit BEFORE the game even starts.

You set these limits (called "hyperparameters") before training even begins:

`max_depth` — "How many questions deep can we go?"

Depth 0 = the root (before any question)

Depth 1 = after the first question

Depth 2 = after the second question

etc.

If you set `max_depth = 3`, the tree is physically not allowed to ask a 4th-level question, no matter how helpful it might seem. It just stops there, even if the leaf still has a mix of Yes/No students inside.

Beginner example: With `max_depth = 3`, the tree might stop at: "CGPA $\leq$ 7.15? $\rightarrow$ Backlogs = Yes? $\rightarrow$ STOP (max depth hit)" — even though this leaf still has 2 Yes + 3 No mixed inside. It just accepts that leaf as "good enough" rather than digging further.

`min_samples_split` — **"Do I even have enough students here to bother splitting?"**

This checks the **PARENT node** *before* even attempting a split. Example: if a node has only 3 students left and `min_samples_split = 4`, the tree isn't even allowed to try splitting this node — it must stop here immediately, whether the node is pure or not.

`min_samples_leaf` — **"Would the resulting groups be too tiny?"**

This checks the **CHILD nodes** *after* a candidate split is proposed. Example: if a candidate split would create one leaf with only 1 student and another with 7, and `min_samples_leaf = 2`, this specific split gets REJECTED (because $1 < 2$) — even if the parent had plenty of data overall. The tree must try a different split instead.

These two sound similar — here's the clearest way to remember the difference:

`min_samples_split` asks the question BEFORE splitting: "Is there enough here to even try?" `min_samples_leaf` asks the question AFTER splitting: "Are the resulting pieces big enough to trust?"

`max_leaf_nodes` — **"How many final answer-boxes can the whole tree have, total?"**

If `max_leaf_nodes = 5`, then no matter how the tree is shaped, it can only ever end up with 5 leaves total across the ENTIRE tree.

Why tiny leaves are risky

Imagine two leaves:

Leaf A: 40 students, all Placed. The prediction "Placed" here is backed by strong, substantial evidence.

Leaf B: just 1 student, who happened to be Placed. The prediction "Placed" here is based on literally ONE example — it might just be a fluke/coincidence, not a real pattern.

Leaf B's rule might not hold true at all for the next new student, even though it "worked" for the training data. This is why we often force `min_samples_leaf` to be more than 1.

Pre-pruning: Pros and Cons

✅ Advantages

Fast to train (fewer splits to try)

Produces a smaller, easier-to-read tree

⚠️ Risks

Stopping TOO early $\rightarrow$ underfitting

Leaves may remain mixed/impure

✅ Advantages

⚠️ Risks

Easier for humans to interpret

Important clarification: Pre-pruning does NOT require every leaf to be perfectly pure. A leaf can stop with mixed data inside (like 2 Yes + 3 No) if a limit (like max_depth) forces it to stop early — the tree just does its best guess with what it has.

3.5 Post-Pruning: Grow BIG First, Then Simplify

Instead of restricting the tree from the start, this approach lets the tree grow as large as it wants (even overfitting on purpose!), and THEN goes back afterward and cuts away the weak/unhelpful branches.

Analogy: Instead of trimming a bonsai tree as it grows, you let a full tree grow wild first, then go through with garden shears and cut away the branches that aren't pulling their weight.

✅ Advantages

⚠️ Costs

Can inspect deeper structure before deciding what's useful (sometimes a branch that looks weak actually leads somewhere good)

Takes more computation (you had to grow the whole big tree first)

The decision to remove a branch is based on an actual measurable criterion

Requires extra math/validation to decide what to trim

Pre-Pruning

Post-Pruning

When applied? During growth

After the tree is fully grown

Basic idea Stop early

Grow big, then simplify

Main risk Might underfit

More computationally expensive

Example tools max_depth, min_samples_leaf Cost-Complexity Pruning

3.6 Cost-Complexity Pruning (CCP) — The Formal Way to Post-Prune

This is the most important formula in Session 3. The idea: **don't just look at accuracy — also charge a "price" for every extra leaf the tree has.** A tree only "deserves" to keep an extra leaf if that leaf earns its keep by meaningfully improving accuracy.

$$R_{\alpha}(T) = R(T) + \alpha|T|$$

Let's break down every single symbol, one at a time:

Symbol	Plain English meaning
T	"This particular tree" (a candidate tree we're evaluating)
$R(T)$	The tree's ERROR — how badly it fits the data (lower is better fit)
$ T $	The NUMBER OF LEAVES in the tree — this represents complexity
α (alpha)	A penalty "price tag" you choose — how much you're willing to "pay" per extra leaf
$R_{\alpha}(T)$	The FINAL score — error PLUS the complexity penalty. We want this number as LOW as possible.

Simple analogy: Imagine you're grading two students' essays. One is a beautifully argued 2-page essay, and the other is a slightly less polished 1-page essay. If you also care about conciseness (penalizing extra length), the shorter essay might actually win the overall grade even though it's a little less "accurate" — because it earns fewer "length penalty" points. That's exactly what α does here: it's a tunable knob for "how much do I care about keeping the tree small?"

If $\alpha = 0 \rightarrow$ no penalty at all for extra leaves $\rightarrow$ you'd always just pick the biggest, most detailed tree (basically no pruning happens).

If α is **LARGE** $\rightarrow$ heavy penalty per leaf $\rightarrow$ you'll strongly prefer small, simple trees, even if they're a bit less accurate.

Worked Example: Comparing 3 candidate trees, with $\alpha = 0.02$

Tree	Leaves $ T $	Error $R(T)$	Penalized Cost Calculation	Final Cost
A — Large	8	0.05	$0.05 + 0.02(8)$	0.21
B — Medium	5	0.08	$0.08 + 0.02(5)$	0.18 ★ WINNER
C — Small	3	0.14	$0.14 + 0.02(3)$	0.20

Let's understand WHY Tree B wins, step by step:

Tree A has the LOWEST raw error (0.05 — it fits the training data best of all three), BUT it has 8 leaves, so it gets penalized a lot: $0.02 \times 8 = 0.16$ added on top. Final: 0.21.

Tree C is much simpler (only 3 leaves, tiny penalty of $0.02 \times 3 = 0.06$), but its raw error is quite high (0.14) — it's TOO simple and is missing real patterns. Final: 0.20.

Tree B sits in the middle: a moderate error (0.08) and a moderate penalty ($0.02 \times 5 = 0.10$). Final: **0.18 — the lowest total, so it wins.**

🌀 **The big beginner lesson here:** *The tree with the lowest training error does NOT automatically win.* You have to balance "how well does it fit" against "how complicated is it." Tree B isn't the most accurate OR the simplest — it's the best **overall trade-off**, which is exactly what cost-complexity pruning is designed to find.

3.7 Weakest-Link Pruning — HOW Do We Actually Find Which Branches to Cut?

Once you've grown a big tree, you need an actual algorithm to decide which branches are the "weakest" (least worth keeping) so you can prune them one at a time.

$$g(t) = \frac{R(t) - R(T_t)}{|T_t| - 1}$$

Symbol

Meaning

$R(t)$ The error IF you collapsed this entire subtree down into just ONE single leaf

$R(T_t)$ The error of the subtree exactly as it currently is (with all its splits still in place)

$|T_t|$ How many leaves are currently inside this subtree

$g(t)$ "Extra error you'd pay, per leaf you'd remove, if you collapsed this subtree"

In plain English: $g(t)$ answers: "If I chopped this branch down to a single leaf, how much accuracy would I lose, divided by how many leaves I'd be getting rid of?" A LOW $g(t)$ means: "collapsing this branch barely hurts accuracy, but saves several leaves — great trade, cut it!" A HIGH $g(t)$ means: "collapsing this branch would hurt accuracy a lot for the leaves saved — don't cut it, it's valuable."

The algorithm, step by step:

Start with your big, fully-grown tree.

Calculate $g(t)$ for every possible subtree.

Find whichever subtree has the SMALLEST $g(t)$ — this is the "weakest link" (the least valuable branch).

Collapse (prune) that one subtree into a single leaf.

Now you have a slightly smaller tree. Repeat the whole process again on this new, smaller tree.

Keep going, producing progressively smaller and smaller trees, each one slightly simpler than the last.

This creates a whole SEQUENCE of trees, from the giant original one down to just a single leaf, each one linked to a specific α value.

You don't just guess a good α — you TEST different values and see which actually performs best, using a systematic process:

Generate candidate α values (e.g., 0, 0.01, 0.02, 0.05, 0.1, ...)

Train a candidate pruned tree for EACH α value (small $\alpha \rightarrow$ bigger tree; large $\alpha \rightarrow$ more heavily pruned/smaller tree)

Evaluate each candidate tree on validation data (data NOT used for training — a fair test)

Pick whichever α gave the BEST validation performance

Evaluate your final chosen tree on a separate held-out test set, just to get one last honest check

In code, this looks like setting a parameter called `ccp_alpha` when creating your `DecisionTreeClassifier` — small `ccp_alpha` $\rightarrow$ larger tree; large `ccp_alpha` $\rightarrow$ stronger pruning.

3.9 Putting It All Together: Pre-Pruning vs. Cost-Complexity Pruning

Aspect	Pre-Pruning	Cost-Complexity Pruning
Timing	During growth	After growth (or through the α "pruning path")
Main knobs	max_depth, min_samples_split, min_samples_leaf	ccp_alpha
Strategy	"Stop early"	"Trade off fit vs. size"
Risk if overdone	Underfitting	Over-pruning (also underfitting)

The three-way spectrum to always remember:

Too little pruning  **Appropriate pruning** Too much pruning

Very deep tree Balanced, useful complexity Very simple tree

High variance Good generalization High bias

Overfitting Best predictions on new data **Underfitting**

3.10 Session 3 Summary (Quick Recap)

A very large/deep tree fits training data almost perfectly, but that's often a bad sign — it may have just memorized noise (overfitting).

Judge trees by performance on NEW, unseen data — never by training accuracy alone.

Pre-pruning = stopping the tree early using limits like `max_depth`, `min_samples_split`, `min_samples_leaf`, `max_leaf_nodes`.

Post-pruning = growing a big tree first, then trimming weak branches afterward, most formally via **Cost-**

Complexity Pruning: $R_\alpha(T) = R(T) + \alpha|T|$.

α controls the strength of the "penalty per leaf" — bigger α = more aggressive pruning = simpler tree.

Weakest-link pruning is the algorithm that finds which specific branch to cut next, by computing $g(t)$ for every subtree and removing the smallest one.

You choose the best α by testing several candidates against validation data — never by guessing.

SESSION 4: Bagging and Feature Subsampling

Session 3 was all about making ONE tree as good as possible. Session 4 asks a totally different question: **what if, instead of perfecting one tree, we just built a BUNCH of different trees and let them vote?**

4.1 The Core Motivation

Analogy: Asking one friend for advice vs. asking a group of friends.

If you ask just ONE friend "should I take this job offer?", their advice depends heavily on their own personal experiences and biases — it might be great advice, or it might be quite skewed. But if you ask 10 different friends (with different backgrounds and perspectives) the same question, and then go with whatever the MAJORITY of them say, your final decision is usually much more reliable — because their individual quirks and blind spots tend to cancel each other out.

This is exactly the idea behind combining multiple decision trees:

One tree	Multiple (diverse) trees	Combined prediction
Can depend TOO heavily on its exact training sample. High "variance risk" (remember this from Session 3!).	Each tree learns from slightly different data or a different subset of features. They tend to make DIFFERENT mistakes from each other.	Voting (classification) or averaging (regression) smooths out individual errors, making the final answer more stable and trustworthy.

📌 **Key line to remember: "Bagging is an ensemble approach mainly designed to reduce variance."** (An "ensemble" just means "a group of models working together.")

4.2 Why NOT Just Train the Same Tree Algorithm Multiple Times?

You might think: "Easy, just run the tree-building algorithm 5 times and combine the results!" But here's the catch — **if you give the exact same data, exact same features, and exact same settings to the algorithm every time, it will build the EXACT SAME tree every single time** (since it's deterministic — no randomness involved). You'd just end up with 5 identical copies of "Tree A," which gives you ZERO extra benefit from combining them.

Analogy: Asking the same one friend the same question 5 times in a row doesn't give you 5 independent opinions — it just gives you the same opinion repeated 5 times. You need genuinely DIFFERENT friends (or genuinely different information) to get real diversity of opinion.

So Session 4 introduces TWO separate tricks to force real diversity between trees:

Bagging → give each tree a slightly different set of TRAINING ROWS (students).

Feature Subsampling → give each tree access to only a random SUBSET of the COLUMNS (features) at each decision point.

4.3 Bagging = "Bootstrap Aggregating" (a fancy name, simple idea)

The word "Bagging" is actually short for **Bootstrap Aggregating**. Let's unpack both halves:

Part 1: "Bootstrap" — creating a new dataset by drawing WITH replacement

What "with replacement" means, explained super simply:

Imagine you have 14 name-tags in a hat, one for each student (S1 through S14). You're going to draw 14 tags to build a new "bootstrap sample." BUT — after each draw, you put that tag BACK into the hat before drawing again. This means:

The SAME student could get drawn multiple times (repeats!)

Some students might never get drawn at all (omitted!)

This is the opposite of your normal intuition of "randomly picking without repeats" — here, repeats are not just allowed, they're expected.

Concrete worked example (from the slides):

Say we draw 14 times for "Tree 1's" bootstrap sample, and we get:



S1, S2, S2, S4, S6, S7, S9, S10, S10, S11, S12, S14, S3, S5

Notice:

S2 appears TWICE (drawn, put back, drawn again) — a "repeated" student.

S10 appears TWICE too — another repeat.

S8 and S13 don't appear AT ALL — they were simply never drawn in these 14 tries. These are called "omitted" students for this particular tree.

Even though we drew 14 times (matching the original dataset's size of 14), because of the repeats-and-omissions, this "bootstrap sample" is actually a bit different in COMPOSITION from the original full dataset.

Different trees get different bootstrap samples, purely by chance:

Tree	Repeated students	Omitted students
Tree 1	S2, S10	S8, S13
Tree 2	S1, S6, S14	S2, S4, S10
Tree 3	S4, S8, S13	S1, S6, S12

See how each tree's sample looks slightly different? That's the WHOLE POINT — this is our first source of randomness/diversity.

 **Key fact to remember:** A bootstrap sample always has the SAME NUMBER of draws as the original dataset (14 draws for our 14 students) — but it does NOT necessarily contain the same number of UNIQUE students, because of the repeats.

Part 2: "Aggregating" — combining everyone's predictions at the end

Once you have multiple trees (each trained on its own slightly-different bootstrap sample), you combine ("aggregate") their individual predictions into one final answer.

The full Bagging recipe, in 3 steps:

Bootstrap: Create several different training samples (each with replacement, same size as original).

Train: Build ONE separate decision tree on EACH bootstrap sample. (Same tree-building algorithm each time — from Sessions 1–3 — just fed different data.)

Aggregate: Combine all the trees' predictions into one final answer (vote or average).

One bootstrap sample → one tree, and why the trees end up DIFFERENT

Even though every tree uses the EXACT SAME algorithm and settings, because each one is trained on a slightly different bootstrap sample of students, they naturally end up structurally different:

- Different class proportions inside their nodes

- Different candidate splits look "best" to each tree

- Different threshold numbers get chosen

- The overall SHAPE of the tree can differ

Important clarification: The learning ALGORITHM is identical across every tree. It's only the underlying TRAINING DATA (the specific bootstrap sample) that differs from tree to tree.

4.4 How Do We Combine ("Aggregate") All the Trees' Predictions?

This depends on whether we're doing classification or regression — same distinction from Session 1!

For classification: Majority Voting

Every tree makes its own individual prediction, and whichever answer gets the MOST votes wins.

Worked Example: Say we have 5 trees, and for one particular new student, they each predict:

Tree Prediction

Tree 1 Yes

Tree 2 No

Tree 3 Yes

Tree 4 Yes

Tree 5 No

Count the votes: **Yes = 3 votes, No = 2 votes.**

Final ensemble prediction: Placed = Yes (majority wins — just like a simple election).

For regression: Averaging

If we're predicting a NUMBER (like salary), we can't "vote" — instead we just take the mathematical average of everyone's individual prediction.

Worked Example: Say we have 3 trees predicting a student's salary package:

Tree Predicted Salary

Tree 1 6.2 LPA

Tree 2 7.0 LPA

Tree 3 6.8 LPA

$$\hat{y} = \frac{6.2 + 7.0 + 6.8}{3} = \boxed{6.67 \text{ LPA}}$$

Final ensemble prediction: 6.67 LPA.

4.5 Feature Subsampling: The SECOND Source of Diversity

Here's a subtle problem with bagging alone: even though each tree sees slightly different training ROWS, if ONE feature (say, CGPA) is by far the single strongest predictor in the whole dataset, then almost EVERY tree — no matter which bootstrap sample it got — is still likely to pick CGPA as its very first (root) question, because it's just so obviously the best choice every time. This means all our trees might STILL end up looking fairly similar near the top, which defeats some of the purpose of building multiple trees.

The fix: at each individual split (not just at the very start, but EVERY time a tree wants to ask a question), we don't let it consider ALL the features — we randomly restrict it to only look at a small random SUBSET of the available features.

Worked Example

Suppose we have 4 total features (CGPA, Internships, Backlogs, Aptitude), and we set `max_features = 2` (meaning: at each split, the tree can only consider 2 randomly-chosen features out of the 4).

At ONE particular split, the randomly chosen subset might be: {CGPA, Aptitude} — so the tree can ONLY choose between these two, even if Internships would have been even better. It picks the best of what's ALLOWED.

At a DIFFERENT split (even in the same tree, or in a different tree), the random subset might instead be: {Internships, Backlogs}.

The tree always picks the best split — but only from whatever features happen to be made available to it at that specific moment.

Why this actually creates more diverse, useful trees

	Without feature subsampling	With feature subsampling
Situation	CGPA is available at every single split, every single time	CGPA sometimes simply isn't offered as an option
Result	Nearly every tree keeps choosing CGPA at the root → similar root splits → similar-looking trees overall	Different trees are FORCED to sometimes rely on other, previously-overlooked features

Concrete worked example:

Tree Features made available at a split Feature the tree ends up choosing

Tree 1 CGPA, Aptitude	CGPA
Tree 2 Internships, Backlogs	Backlogs
Tree 3 CGPA, Internships	Internships
Tree 4 Backlogs, Aptitude	Aptitude

Notice how each tree ends up leaning on a genuinely DIFFERENT feature — this is exactly the diversity we wanted! If CGPA had been available every single time, Trees 2, 3, and 4 probably would have all picked CGPA too, making them far more similar to each other than they are now.

4.6 Two Independent Sources of Randomness — Side by Side

	Bagging	Feature Subsampling
What gets randomized?	Which training ROWS (students) each tree sees	Which COLUMNS (features) are available at each split
Mechanism	Bootstrap sampling (sample with replacement)	Randomly restrict candidate features at each split
What it creates	Different training SETS per tree	Different feature "VIEWS" per tree/split
Main purpose	Reduces variance (less dependence on any one specific training sample)	Reduces correlation between trees (prevents them all looking too similar)
Used in Random Forest?	☒ Yes	☒ Yes

A well-built ensemble benefits when its individual trees are each reasonably accurate on their own, but genuinely DIFFERENT from one another — not carbon copies. Both bagging and feature subsampling are tools purely for creating that healthy diversity.

4.7 The Payoff: This IS a Random Forest!

Everything in Session 4 has been building toward this one formula:

Random Forest = Bagging + Random Feature Subsampling + Many Decision Trees + Aggregation

Putting it all together, step by step:

Create many different bootstrap samples of the training data (with replacement).

Train one decision tree on each bootstrap sample (using everything from Sessions 1–3: greedy splitting via entropy/Gini, with sensible pruning limits).

At EVERY split inside EVERY tree, only let it consider a random subset of the available features.

Once all the trees are trained, combine their predictions: majority vote for classification, average for regression.

Why does this work so well in practice? Because each individual tree is trained differently (different rows AND different feature availability), their individual mistakes tend to be somewhat random and independent of each other — like the friends-giving-advice analogy from earlier. When you combine many "somewhat different, somewhat wrong" opinions together, the ERRORS tend to cancel each other out, while the genuinely useful PATTERNS (which most trees will agree on) get reinforced. The end result is usually much more accurate and stable than any single tree could be on its own.

4.8 A Sneak Peek at What Comes Next

The slides mention (but don't fully cover) a concept for a future session: **Out-of-Bag (OOB) evaluation**.

Remember how each bootstrap sample OMITS some students (like S8 and S13 for Tree 1)? Those omitted/left-out students are called "out-of-bag" for that particular tree. Since Tree 1 never saw S8 during training, we can actually use S8 as a free, built-in "test" for Tree 1 — checking how well it predicts a student it genuinely never trained on. This gives you a clever way to estimate how well your Random Forest will perform on new data, without needing to set aside a separate validation set at all!

4.9 Session 4 Summary (Quick Recap)

A single tree can be too dependent on its exact training data (high variance) — combining several DIVERSE trees helps smooth this out.

Training the same algorithm on the same data repeatedly gives you IDENTICAL trees — no benefit. You need genuine diversity.

Bagging = create multiple "bootstrap samples" (draw with replacement, same size as original) and train one tree per sample. Some students get repeated, some get omitted, per tree.

Aggregation: classification uses majority voting; regression uses averaging.

Feature Subsampling = at each split, only let the tree consider a random SUBSET of features (not all of them) — this forces trees to rely on different features, increasing diversity further.

Bagging randomizes ROWS; Feature Subsampling randomizes COLUMNS. Both exist purely to create healthy diversity among trees.

Random Forest = Bagging + Feature Subsampling + Many Trees + Aggregation.

THE COMPLETE JOURNEY: How All 4 Sessions Connect

Think of it as a story with a clear beginning, middle, and end:

Session 1 (The Foundation): "Here's what a decision tree even IS — a chain of yes/no questions built by greedily picking the best question at each step, splitting mixed groups into purer ones, until we reach pure leaves."

Session 2 (The Math): "Here's EXACTLY how the tree decides which question is 'best' — using Entropy/Information Gain, or Gini Impurity, or MSE for numbers — always picking whichever split reduces confusion the most."

Session 3 (The Fix for Overfitting): "If you let a tree grow unchecked using Session 1 & 2's rules, it can get TOO good at memorizing the training data and fail on new data. Pruning (pre-pruning limits, or formal cost-complexity pruning) reins this in to find the sweet spot between too-simple and too-complex."

Session 4 (The Power-Up): "Instead of relying on just ONE carefully-tuned tree, why not build MANY diverse trees (using bootstrap sampling + feature subsampling) and let them vote together? This is the foundation of Random Forests, which are typically far more accurate and stable than any single tree."

End of guide. If any section still feels unclear, tell me exactly which part (e.g., "the entropy formula" or "why min_samples_split and min_samples_leaf are different") and I'll re-explain it with an even simpler example.